

1. Algoritmos Implementados

1.1 Dijkstra Clássico

O algoritmo de Dijkstra resolve o problema de caminho mais curto de fonte única explorando iterativamente os vértices em ordem crescente de distância mínima conhecida. Utiliza uma fila de prioridade (min-heap) para selecionar eficientemente o próximo vértice a processar, garantindo que cada vértice seja processado apenas uma vez com sua distância mínima definitiva. Sua complexidade de tempo é $O((E + V) \log V)$, onde E é o número de arestas e V , o de vértices. Opera com pesos em ponto flutuante para máxima precisão.

1.2 Dijkstra Bidirecional

Esta variante executa duas buscas simultâneas: uma partindo da origem (S) em direção ao destino (D), e outra partindo de D "para trás" em direção a S . A busca termina quando as fronteiras se encontram ou quando a soma das menores distâncias em ambas as filas excede o menor caminho completo já encontrado. A intuição é que, ao explorar em ambas as direções, podemos encontrar o caminho mais rapidamente, especialmente em grafos onde o caminho entre S e D não requer explorar grande parte do grafo. Sua complexidade de tempo é $O((E + V) \log V)$ no pior caso, mas com constantes menores e melhor desempenho médio em grafos esparsos em comparação à implementação clássica, pois explora aproximadamente metade do espaço de busca.

1.3 Dial Dijkstra

A otimização de Dial substitui a fila de prioridade por uma estrutura de *buckets*, onde cada *bucket* $b[i]$ armazena vértices com distância exatamente i da origem. Esta abordagem é eficiente quando os pesos das arestas são pequenos valores inteiros. Para este serviço, os pesos são arredondados para inteiros, sacrificando precisão em favor de velocidade. Sua complexidade é $O(E + V + V \times C)$, onde C é o maior peso de uma aresta. Costuma ser vantajoso quando $C \ll V \log V$, típico de grafos densos com pesos pequenos.

1.4 Dijkstra Bidirecional com Dial

Combina as estratégias bidirecional e de Dial, executando buscas simultâneas e utilizando estruturas de *buckets* em ambas as direções. Complexidade: $O(E + V + V \times C)$. Teoricamente eficiente quando tanto a busca bidirecional quanto buckets são vantajosos.

1.5 Programação Linear

O problema de caminho mais curto pode ser modelado como um problema de fluxo de custo mínimo. Seja $x_{\{u,v\}}$ uma variável contínua no intervalo $[0,1]$ indicando o fluxo na aresta (u,v) . Função objetivo: minimizar $\sum w_{\{u,v\}} \cdot x_{\{u,v\}}$, onde $w_{\{u,v\}}$ é o peso da aresta. Restrições:

- Conservação de fluxo: Para cada vértice v (exceto S e D), $\sum x_{\{u,v\}} - \sum x_{\{v,w\}} = 0$;
- Fonte: $\sum x_{\{S,v\}} - \sum x_{\{u,S\}} = 1$; Destino: $\sum x_{\{u,D\}} - \sum x_{\{D,v\}} = -1$.

Esta formulação garante que, exatamente, uma unidade de fluxo saia de S e chegue em D , e as restrições de conservação asseguram um caminho válido. Otimizações implementadas: Para melhorar a eficiência computacional, utilizou-se: (1) estruturas de adjacência pré-processadas evitando iteração repetida sobre todas as arestas; (2) dicionários para acesso rápido às variáveis; (3) método Simplex demonstrou ser significativamente bom para este tipo específico de problema de fluxo em grafos esparsos. Estas otimizações reduziram a complexidade prática de construção do modelo de $O(V \times E)$.

para $O(V + E)$. Complexidade: O método Simplex tem complexidade exponencial no pior caso, mas para problemas de fluxo em redes esparsas, comporta-se de forma polinomial.

2. Comparação e Resultados

2.1 Precisão: Algoritmos Exatos vs Aproximados

Os algoritmos baseados em Dijkstra Clássico, Dijkstra Bidirecional e Programação Linear operam com pesos em ponto flutuante e são exatos, sempre encontrando o caminho ótimo. Já os algoritmos Dial arredondam pesos para inteiros, tornando-se aproximados. Tomando os resultados do teste 6 ($V = 86$, $E = 463$) como exemplo para análise:

- Algoritmos Exatos: 48.787 (Dijkstra Clássico, Bidirecional, PL);
- Algoritmos Dial: 49.0 (na versão clássica e na bidirecional);
- Diferença: 0.213 (0.44%).

Esta diferença é consistentemente pequena ($< 1\%$) em todos os testes, validando que os algoritmos de Dial são adequados para o serviço de velocidade extrema, onde uma margem de erro de $\sim 0.5\%$ é aceitável em troca de maior velocidade de processamento.

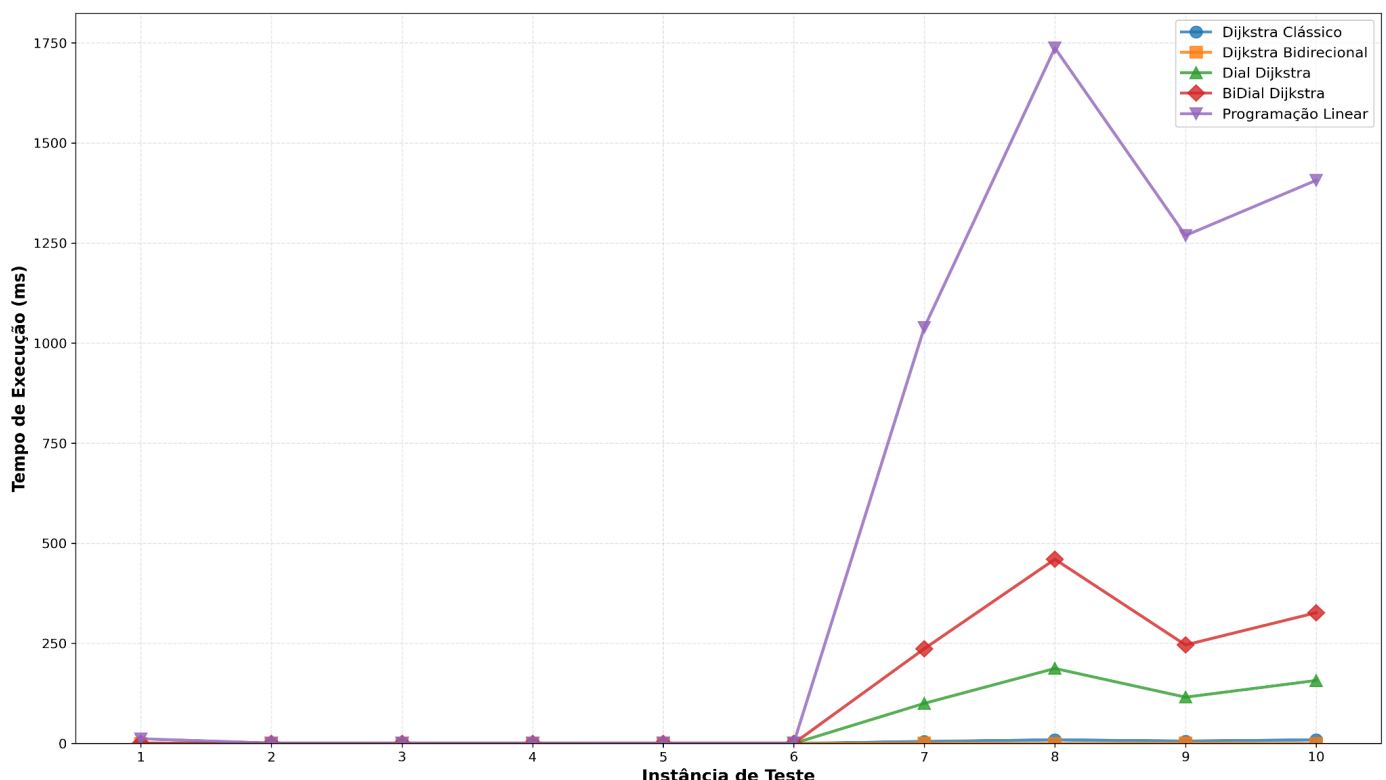
2.2 Análise de Desempenho: Teoria vs Prática

Expectativa teórica (considerando as complexidades explicitadas na seção 1):

- **Dijkstra clássico:** bom em grafos esparsos. Custo dominado pela quantidade de relaxamentos;
- **Dijkstra bidirecional:** mesmo limite assintótico, mas costuma reduzir a busca para, aproximadamente, metade na prática quando frentes encontram-se cedo;
- **Dial (buckets):** muito rápido se C (maior peso inteiro) for pequeno; no entanto, tem desempenho fortemente degradado para C grande;
- **Dial bidirecional:** herda vantagens da bidirecionalidade e limitações de Dial;
- **Programação Linear:** apesar do número de variáveis e de restrições ser linear no tamanho da entrada e o método simplex ser eficiente em redes esparsas, raramente essa abordagem seria capaz de superar algoritmos especializados em tempo.

Dados Experimentais:

Comparação de Desempenho: Todos os Algoritmos em Todas as Instâncias



Algoritmo	Teste 1 (us)	Teste 2 (us)	Teste 3 (us)	Teste 4 (us)	Teste 5 (us)	Teste 6 (us)	Teste 7 (us)	Teste 8 (us)	Teste 9 (us)	Teste 10 (us)
Dijkstra Clássico	0	0	0	0	0	0	5000	9017	6008	9019
Dijkstra Bidirecional	1	1	1	1	1	1	1005	1	1000	1
Dial Dijkstra	0	0	0	0	0	0	100048	187245	115516	157176
BiDial Dijkstra	656	0	0	0	0	0	236625	460072	246006	326423
Programação Linear	11604	1019	1105	1044	1175	1214	1039224	1737242	1268840	1406558

Análise dos resultados

- Em instâncias pequenas (testes 1 a 6) todos os métodos são muito rápidos;
- A partir do teste 7 ($V \approx 61k$, $E \approx 547k$) começa-se a ver as diferenças de desempenho;
- De 8 a 10 (70-89k vértices) o padrão se mantém: bidirecional consistentemente sub-ms a poucos ms; Dial/BiDial variam muito; PL fica ~ 1.3 – 1.8 s;

Possíveis causas para os resultados observados

- Encontro precoce das frentes (bidirecional): nas instâncias grandes o caminho $S \leftrightarrow D$ atravessa “corredores” relativamente restritos. Assim, a bidirecionalidade explora muito menos vértices, explicando o ganho prático ($\ll$ fator teórico).
- Dependência de C (Dial): as primeiras linhas dos grandes arquivos mostram pesos na casa das centenas. Dessa forma, C é grande e Dial varre muitos buckets vazios $\rightarrow$ custo dominado por $V \times C$. BiDial adiciona *overhead* de coordenação/duplicação.
- Sobrecarga do modelo LP: modelar cada aresta orientada gera $\approx 2E$ variáveis. Montar e resolver esse modelo impõe fatorizações e iterações que crescem muito mais rápido que relaxamentos simples de Dijkstra.
- Estrutura do grafo e distribuição de pesos: grafos esparsos e caminhos concentrados favorecem bidirecionalidade. Grafos com pesos grosseiramente espalhados e alto C penalizam estruturas por buckets.

2.3 Adequação aos Serviços Propostos

- **Serviço de Alta Precisão** - Recomendação principal: Dijkstra Bidirecional. Motivo: precisão exata combinada com tempo extremamente baixo nas instâncias testadas (tipicamente < 5 – 10 ms). Para tarefas offline que exigem exatidão e rapidez na geração de múltiplas rotas, sua robustez e velocidade tornam-no a escolha natural;
- **Serviço de Velocidade Extrema** - Recomendação: Dijkstra Clássico ou Bidirecional. Motivo: ambos entregam latência muito baixa e são simples de integrar. Dial só é adequado se for possível garantir pesos inteiros e pequenos; caso contrário, seu comportamento prático tem desempenho abaixo das implementações clássicas;
- **Programação Linear** - Uso recomendado: validação offline. Motivo: PL, otimizada com Simplex e poda prévia, torna-se viável (~ 1.4 – 1.8 s para instâncias maiores) para checagem de resultados, mas é muito mais lenta que algoritmos especializados.

2.4 Conclusão

Os experimentos mostram que validações teóricas nem sempre se traduzem em ganhos práticos: embora Dial pareça atraente teoricamente, sua dependência direta do parâmetro C (maior peso na instância) e o custo de varredura de buckets o tornaram ineficaz para algumas das instâncias testadas. A combinação bidirecional com Dial parece ter amplificado sobrecargas (*overhead*) e piorou o desempenho.

O Dijkstra Bidirecional confirmou-se a solução mais equilibrada: exatidão e latência mínima em quase todas as instâncias testadas, tornando-o a recomendação pragmática para ambos os serviços (alta precisão e baixa latência). A Programação Linear, apesar de ser uma ferramenta poderosa e viável, continua imprópria para produção em tempo real.

Algumas recomendações finais concretas seriam:

- Adotar Dijkstra Bidirecional como algoritmo principal para produção;
- Reservar a otimização de Dial para cenários com pesos pequenos ($C \ll V \log V$);
- Usar PL pode ser útil para validação/auditoria *offline*.